

Shell Scripting: A Complete Mastery Guide

Section 2.0 — From Zero to Expert in Bash Shell Scripting

"A shell script is a computer program designed to be run by a Unix shell. It automates tasks that would otherwise be performed manually by a system operator."



PREREQUISITES CHECKLIST

Before starting Section 2, ensure you have:

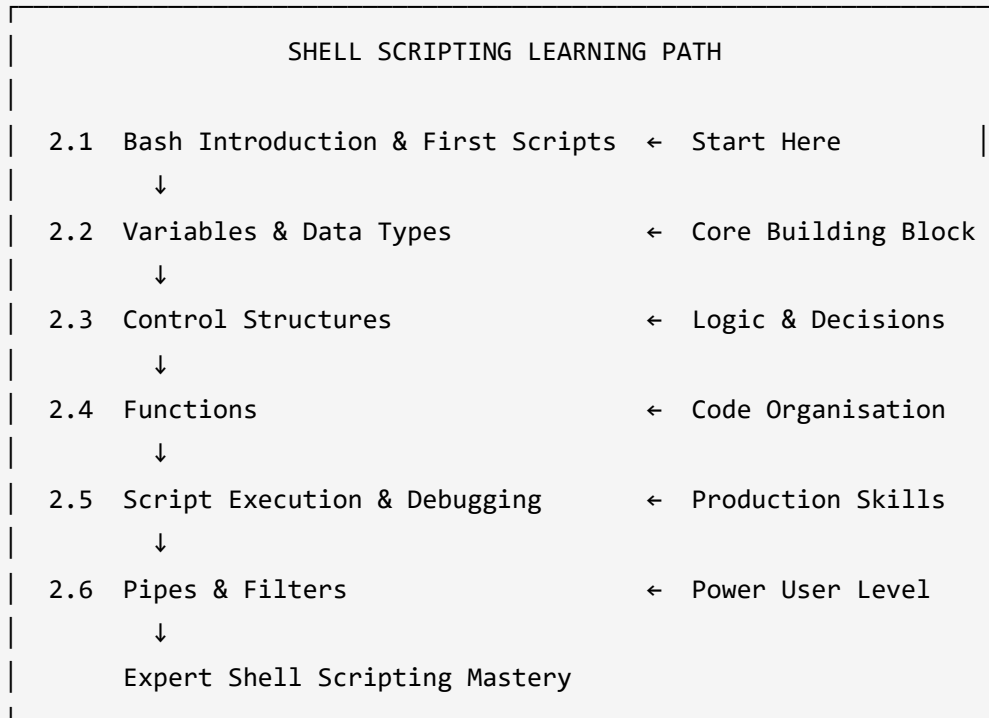
- ☒ Completed Section 1 — Linux Systems (Files, Permissions, Processes)
- ☒ Comfortable with basic terminal commands (`ls` , `cd` , `grep` , `cat`)
- ☒ A Linux terminal open and ready
- ☐ Text editor installed (`nano` , `vim` , or `VSCode`)
- ☐ Basic understanding of file permissions (`chmod`)

Verify your bash version:

```
bash --version
# GNU bash, version 5.1.16(1)-release (x86_64-pc-linux-gnu)
# Bash 4.0+ recommended for all features in this guide
```



SECTION 2 LEARNING ROADMAP





QUICK START — YOUR FIRST SCRIPT IN 60 SECONDS

```
# Create your first script
cat > ~/hello.sh << 'EOF'
#!/bin/bash
echo "Hello, Shell Scripting World!"
echo "Today is: $(date)"
echo "You are: $(whoami)"
EOF

# Make it executable
chmod +x ~/hello.sh

# Run it
~/hello.sh
```

Expected output:

```
Hello, Shell Scripting World!
Today is: Mon Jan 15 10:30:00 UTC 2024
You are: alice
```

You just wrote and ran your first bash script. Let's go deeper.

Section 2.1: Bash Shell Scripting

Learning Objectives

- Understand what a shell script is and why it matters
- Write correctly structured bash scripts with shebangs
- Execute scripts multiple ways and understand the differences
- Use comments, whitespace, and formatting for readable code
- Understand how bash interprets and processes scripts

Introduction to Bash Scripting

Shell scripting is the art of automating command-line workflows. Every task you do in the terminal repeatedly is a candidate for a script. System administrators use scripts to manage hundreds of servers simultaneously. DevOps engineers use them to automate deployments. Data scientists use them to process files in bulk.

Bash (**B**ourne **A**gain **S**hell) is the default shell on most Linux distributions and macOS. It's a superset of the original `sh` (Bourne Shell), adding powerful features for interactive use and scripting. When you script in bash, you're writing in a language that is literally built into every major Unix-like system on Earth.

Why Shell Scripting?

TASK	MANUAL WAY	SCRIPT WAY
Rename 500 photos	Click each file 15 minutes	3-line for loop 0.5 seconds
Deploy to 20 servers	SSH + commands × 20 1 hour	1 script, 30 seconds
Daily backup	Remember, manual run Sometimes forgotten	Cron + script Never forgotten
Process CSV file	Open, filter, save Excel, manual	awk/sed pipeline Instant, repeatable

Anatomy of a Bash Script

```
#!/bin/bash

# -----
# Script: my_first_script.sh
# Purpose: Demonstrate script structure
# Author: Alice Smith
# Date: 2024-01-15
# Version: 1.0
# Usage: ./my_first_script.sh [argument]
# -----
```

```
# — SECTION 1: Configuration —
```

```
SCRIPT_NAME="my_first_script"
```

```
SCRIPT_VERSION="1.0"
```

```
# — SECTION 2: Main Logic —
```

```
echo "Script: $SCRIPT_NAME v$SCRIPT_VERSION"
```

```
echo "Running as: $(whoami)"
```

```
echo "Current directory: $(pwd)"
```

```
# — SECTION 3: Cleanup —
```

```
echo "Done!"
```

```
exit 0          # Exit with success code
```

The Shebang Line (#!)

The **shebang** (also called hashbang) is the FIRST line of every script. It tells the OS which interpreter to use:

```
#!/bin/bash      → Use bash at /bin/bash (most common)
#!/usr/bin/env bash → Find bash in PATH (more portable, preferred)
#!/bin/sh        → POSIX sh (compatible with all sh-like shells)
#!/usr/bin/python3 → Python interpreter (not bash, but same concept!)
#!/usr/bin/perl  → Perl interpreter
```

```
# WHY #!/usr/bin/env bash is BETTER:
# On some systems, bash is not in /bin/bash:
#   macOS: /usr/local/bin/bash (Homebrew)
#   NixOS: /nix/store/.../bin/bash
#   FreeBSD: /usr/local/bin/bash
#
# env searches your PATH and finds bash wherever it is
# This makes scripts MORE PORTABLE across systems

# Always use:
#!/usr/bin/env bash

# Instead of:
#!/bin/bash # Fails on systems where bash is elsewhere
```

The `set` Options — Script Safety Settings

```
#!/usr/bin/env bash

# PROFESSIONAL SCRIPTS ALWAYS START WITH THESE:

set -e          # Exit immediately if any command fails
set -u          # Treat unset variables as errors
set -o pipefail # Catch errors in pipelines (cmd1 | cmd2)
set -x          # (Optional) Print each command before executing (debugging)

# Short version combining all three:
set -euo pipefail

# WHY THESE MATTER:
# Without -e:   Script continues after a failed command → chaos!
# Without -u:   Typos in variable names silently use empty string
# Without -pipefail: "false | true" reports success! (false is hidden)
```

Writing and Executing Basic Bash Scripts

Methods of Script Execution

```
# METHOD 1: Direct execution (requires executable permission)
chmod +x script.sh
./script.sh

# METHOD 2: Explicit interpreter call (no permission needed)
bash script.sh

# METHOD 3: Source (runs in CURRENT shell – shares variables!)
source script.sh
. script.sh      # Dot is an alias for 'source'

# METHOD 4: Using env (portable)
env bash script.sh

# KEY DIFFERENCE: ./script.sh vs source script.sh
# ./script.sh → Spawns NEW sub-shell → changes don't affect parent
# source script.sh → Runs IN CURRENT shell → changes DO affect parent

# DEMONSTRATION:
echo "count=42" > /tmp/vars.sh
./script.sh; echo "count=$count"      # Output: count=
source /tmp/vars.sh; echo "count=$count" # Output: count=42
```

Script File Organization

RECOMMENDED PROJECT STRUCTURE:

```
project/
├── bin/
│   ├── deploy.sh      ← Main executable scripts
│   └── backup.sh
├── lib/
│   ├── common.sh      ← Shared functions (sourced by other scripts)
│   └── logging.sh
├── config/
│   └── settings.conf  ← Configuration variables
├── logs/
│   └── .gitkeep
└── README.md
```

Practical Examples

Example 1: System Information Script

```
#!/usr/bin/env bash
# system_info.sh - Display comprehensive system information
set -euo pipefail

# — Color codes for pretty output —————
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
BOLD='\033[1m'
RESET='\033[0m'

# — Helper function for section headers —————
print_header() {
    echo -e "\n${BLUE}${BOLD}== $1 ==${RESET}"
}

print_info() {
    echo -e "  ${GREEN}✓${RESET} $1: ${YELLOW}$2${RESET}"
}

# — Main script —————
echo -e "${BOLD}┌────────────────────────────────────────────────────────────────────────────────┐${RESET}"
echo -e "${BOLD}│          SYSTEM INFORMATION REPORT          │${RESET}"
echo -e "${BOLD}└────────────────────────────────────────────────────────────────────────────────┘${RESET}"
```

```
print_header "Operating System"
print_info "Distribution" "${grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '\\"'}"
print_info "Kernel" "${uname -r}"
print_info "Architecture" "${uname -m}"
print_info "Hostname" "${hostname}"

print_header "Hardware"
print_info "CPU" "${grep 'model name' /proc/cpuinfo | head -1 | cut -d: -f2 | xargs}"
print_info "CPU Cores" "${nproc}"
print_info "Total RAM" "${free -h | awk '/^Mem:/{print $2}'}"
print_info "Free RAM" "${free -h | awk '/^Mem:/{print $4}'}"

print_header "Storage"
df -h | grep -E '^/dev' | while read -r fs size used avail pct mnt; do
    print_info "Mount: $mnt" "$used / $size ($pct used)"
done

print_header "Network"
ip -4 addr show | grep 'inet ' | while read -r line; do
    ip_addr=$(echo "$line" | awk '{print $2}')
    iface=$(echo "$line" | awk '{print $NF}')
    print_info "Interface: $iface" "$ip_addr"
done

print_header "System Load"
print_info "Uptime" "${uptime -p}"
print_info "Load Average" "${cat /proc/loadavg | awk '{print $1, $2, $3}'}"
print_info "Running Processes" "${ps aux | wc -l}"

echo ""
echo -e "${GREEN}Report generated: $(date){RESET}"
```

```
chmod +x system_info.sh  
./system_info.sh
```

Example 2: File Backup Script

```
#!/usr/bin/env bash
# backup.sh – Create timestamped backups of important directories
set -euo pipefail

# — Configuration —————
SOURCE_DIR="${1:-$HOME}"          # Default to home dir, or take from arg
BACKUP_DIR="/tmp/backups"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_NAME="backup_${TIMESTAMP}.tar.gz"
MAX_BACKUPS=5                     # Keep only 5 most recent backups

# — Validation —————
if [[ ! -d "$SOURCE_DIR" ]]; then
    echo "ERROR: Source directory '$SOURCE_DIR' does not exist!" >&2
    exit 1
fi

# — Create backup directory —————
mkdir -p "$BACKUP_DIR"

# — Create the backup —————
echo "📦 Creating backup of: $SOURCE_DIR"
echo "  Destination: $BACKUP_DIR/$BACKUP_NAME"

tar -czf "$BACKUP_DIR/$BACKUP_NAME" \
    --exclude="*.log" \
    --exclude="*.tmp" \
    --exclude=".cache" \
    "$SOURCE_DIR" 2>/dev/null
```

```
SIZE=$(du -sh "$BACKUP_DIR/$BACKUP_NAME" | cut -f1)
echo "  ✅ Backup created: $SIZE"

# — Cleanup old backups (keep only MAX_BACKUPS newest) —————
echo "  🧹 Cleaning old backups (keeping $MAX_BACKUPS newest)..."
BACKUP_COUNT=$(ls -1 "$BACKUP_DIR"/backup_*.tar.gz 2>/dev/null | wc -l)

if [[ $BACKUP_COUNT -gt $MAX_BACKUPS ]]; then
    # Sort by name (timestamp-based), delete oldest
    ls -1t "$BACKUP_DIR"/backup_*.tar.gz | \
        tail -n "+$(MAX_BACKUPS + 1)" | \
        xargs rm -f
    echo "  ✅ Removed $((BACKUP_COUNT - MAX_BACKUPS)) old backup(s)"
fi

echo ""
echo "  📁 Current backups:"
ls -lh "$BACKUP_DIR"/backup_*.tar.gz 2>/dev/null || echo "  (none)"

echo ""
echo "  ✅ Backup complete!"
```

Example 3: Script Template (Professional Starting Point)

```
#!/usr/bin/env bash

# -----
# Script:  template.sh
# Purpose: Professional script template with all best practices
# Usage:  ./template.sh [-v] [-h] [-o output_file] input_file
# -----

# — Safety settings -----
set -euo pipefail
IFS=$'\n\t'    # Set safe Internal Field Separator

# — Script metadata -----
readonly SCRIPT_NAME=$(basename "$0")
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
readonly SCRIPT_VERSION="1.0.0"
readonly LOG_FILE="/tmp/${SCRIPT_NAME%.sh}.log"

# — Default configuration -----
VERBOSE=false
OUTPUT_FILE=""

# — Logging functions -----
log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*" | tee -a "$LOG_FILE"; }
log_info() { log "INFO: $*"; }
log_warn() { log "WARN: $*" >&2; }
log_error() { log "ERROR: $*" >&2; }
log_debug() { [[ "$VERBOSE" == true ]] && log "DEBUG: $*" || true; }

# — Cleanup function (runs even if script exits on error) -----
```

```
cleanup() {  
    local exit_code=$?  
    log_debug "Cleanup running (exit code: $exit_code)"  
    # Remove temp files here  
    # rm -f /tmp/temp_$$_*  
    exit $exit_code  
}  
trap cleanup EXIT
```

```
# — Usage/help function —————
```

```
usage() {  
    cat << EOF  
    ${SCRIPT_NAME} v${SCRIPT_VERSION}
```

USAGE:

```
    ${SCRIPT_NAME} [OPTIONS] input_file
```

OPTIONS:

```
-h, --help          Show this help message  
-v, --verbose        Enable verbose output  
-o, --output FILE    Specify output file
```

EXAMPLES:

```
    ${SCRIPT_NAME} data.txt  
    ${SCRIPT_NAME} -v -o result.txt data.txt
```

```
EOF
```

```
}
```

```
# — Argument parsing —————
```

```
parse_args() {  
    while [[ $# -gt 0 ]]; do
```

```
case "$1" in
    -h|--help)
        usage
        exit 0
        ;;
    -v|--verbose)
        VERBOSE=true
        shift
        ;;
    -o|--output)
        OUTPUT_FILE="$2"
        shift 2
        ;;
    -*)
        log_error "Unknown option: $1"
        usage
        exit 1
        ;;
    *)
        INPUT_FILE="$1"
        shift
        ;;
esac

done

}

# — Main function —————
main() {
    parse_args "$@"

    log_info "Script started: ${SCRIPT_NAME} v${SCRIPT_VERSION}"
    log_debug "Verbose mode: enabled"
```

```
# Validate required arguments
if [[ -z "${INPUT_FILE:-}" ]]; then
    log_error "No input file specified"
    usage
    exit 1
fi

if [[ ! -f "$INPUT_FILE" ]]; then
    log_error "Input file not found: $INPUT_FILE"
    exit 1
fi

log_info "Processing: $INPUT_FILE"
# — YOUR SCRIPT LOGIC GOES HERE —————

log_info "Done!"
}

# — Entry point —————
main "$@"
```



Hands-on Exercise 2.1

```
# EXERCISE: Write and execute your first scripts

# Task 1: Create a "Hello World" script with proper structure
cat > ~/exercise_2_1.sh << 'EOF'
#!/usr/bin/env bash
# exercise_2_1.sh - Hello World with style

echo "[ My First Proper Script! ]"
echo ""
echo "User:    $(whoami)"
echo "Shell:   $SHELL"
echo "Date:    $(date)"
echo "PWD:     $(pwd)"
EOF

# Make executable
chmod +x ~/exercise_2_1.sh

# Run it
~/exercise_2_1.sh

# Task 2: Understand execution methods
# Method 1: Direct
./exercise_2_1.sh 2>/dev/null || bash ~/exercise_2_1.sh

# Method 2: Explicit bash
```

```
bash ~/exercise_2_1.sh

# Method 3: Check if sourcing shares variables
echo 'MY_VAR="shared"' > /tmp/share_test.sh
bash /tmp/share_test.sh
echo "After bash exec: MY_VAR=$MY_VAR"      # Empty!
source /tmp/share_test.sh
echo "After source: MY_VAR=$MY_VAR"          # shared!

# Task 3: Use the professional template to create a script
# that shows disk usage for the top 5 largest directories
```



Common Mistakes in Section 2.1

```
# MISTAKE 1: Forgetting the shebang
# BAD:
echo "Hello"

# GOOD:
#!/usr/bin/env bash
echo "Hello"

# MISTAKE 2: Forgetting to make script executable
./script.sh # Error: Permission denied
# FIX:
chmod +x script.sh

# MISTAKE 3: Windows line endings (\r\n instead of \n)
# Symptom: "bash: ./script.sh: /bin/bash^M: bad interpreter"
# FIX:
dos2unix script.sh
# or: sed -i 's/\r//' script.sh

# MISTAKE 4: Spaces around = in variable assignment
MY_VAR = "value" # WRONG – bash thinks MY_VAR is a command!
MY_VAR="value"   # CORRECT

# MISTAKE 5: Not quoting variables (spaces break things)
FILENAME="my file.txt"
cat $FILENAME    # WRONG – tries to cat "my" and "file.txt"
cat "$FILENAME"  # CORRECT – treats as one argument
```



Best Practices for Bash Scripts

```
# 1. Always start with:
#!/usr/bin/env bash
set -euo pipefail

# 2. Quote ALL variable expansions
echo "$variable"      # Not: echo $variable
ls "$directory"       # Not: ls $directory

# 3. Use [[ ]] not [ ] for conditionals (bash-specific, safer)
if [[ -f "$file" ]]; then    # Not: if [ -f $file ]

# 4. Use $(command) not `backticks` for command substitution
DATE=$(date)           # Not: DATE=`date`

# 5. Use printf over echo for portable output
printf '%s\n' "$message"  # More reliable than echo

# 6. Check if commands exist before using them
if ! command -v jq &>/dev/null; then
    echo "jq is required but not installed" >&2
    exit 1
fi

# 7. Use meaningful exit codes
exit 0  # Success
exit 1  # General error
exit 2  # Misuse of command
```

```
# 8. Always validate user input
if [[ -z "$1" ]]; then
    echo "Usage: $0 <argument>" >&2
    exit 1
fi
```

Section 2.2: Variables and Data Types

Learning Objectives

- Declare, assign, and use variables correctly
- Understand variable scope (local vs global)
- Work with special variables (`$0` , `$1` , `$#` , `$?` , `$$`)
- Use arrays (indexed and associative) effectively
- Apply string operations, arithmetic, and type manipulation

Introduction

Variables are the memory of your script. They store values that your program needs — filenames, counts, user input, command output. Unlike many programming languages, bash is weakly typed — every variable is fundamentally a string, with special mechanisms for integer arithmetic.

Understanding variables deeply — their scoping rules, special forms, and operations — is what separates a casual script writer from a bash expert.

Variable Declaration and Usage

Basic Variable Assignment

```
#!/usr/bin/env bash

# — Variable Assignment Rules —————
# RULE 1: NO spaces around the = sign
NAME="Alice"           # String
AGE=30                 # Number (still stored as string)
IS_ACTIVE=true         # Boolean (just a string "true")

# RULE 2: Reference with $VARIABLE or ${VARIABLE}
echo "$NAME"           # Alice
echo "${NAME}"         # Alice (preferred in complex expressions)
echo "Hello, ${NAME}!" # Hello, Alice!

# RULE 3: Variable names are CASE SENSITIVE
name="lowercase"
NAME="UPPERCASE"
echo "$name"           # lowercase
echo "$NAME"           # UPPERCASE

# RULE 4: Convention: UPPERCASE for constants/environment, lowercase for locals
readonly MAX_RETRIES=3 # Constant (can't be changed!)
readonly PI=3.14159
log_file="/tmp/app.log" # Regular variable

# RULE 5: Unset variables default to empty string (unless set -u)
```

```
echo "Empty: '$UNDEFINED_VAR'" # Output: Empty: ''  
# With set -u: this would cause an error!
```

Variable Expansion Forms

```
#!/usr/bin/env bash
```

```
NAME="Alice"
```

```
EMPTY=""
```

```
# — Basic expansion
```

```
echo "$NAME"           # Alice
```

```
# — Default values
```

```
echo "${NAME:-Unknown}" # Alice (NAME is set)
```

```
echo "${EMPTY:-Unknown}" # Unknown (EMPTY is empty)
```

```
echo "${UNDEFINED:-Unknown}" # Unknown (not set)
```

```
# Assign default AND use it:
```

```
echo "${NAME:=DefaultName}" # Alice (NAME already set, no change)
```

```
echo "${EMPTY:=NewValue}"   # NewValue (EMPTY was empty, now set!)
```

```
echo "$EMPTY"               # NewValue (it was actually assigned!)
```

```
# Error if unset:
```

```
# echo "${REQUIRED_VAR:?Variable REQUIRED_VAR must be set}"
```

```
# → error: Variable REQUIRED_VAR must be set
```

```
# — String operations
```

```
FILE="/home/alice/documents/report.txt"
```

```
# String length
```

```
echo "${#FILE}"          # 38 (character count)
```

```
echo "${#NAME}"           # 5
```

```
# Substring extraction: ${VAR:start:length}
```

```
echo "${FILE:6:5}"          # alice (start at 6, take 5 chars)
echo "${NAME:0:3}"          # Ali (first 3 chars)
echo "${NAME: -3}"          # ice (last 3 chars – note the space!)

# Remove prefix (shortest match)
echo "${FILE#/home/}"       # alice/documents/report.txt
# Remove prefix (longest/greedy match)
echo "${FILE##*/}"          # report.txt (everything after last /)

# Remove suffix (shortest match)
echo "${FILE%.txt}"         # /home/alice/documents/report
# Remove suffix (longest match)
echo "${FILE%*.}"           # /home/alice/documents/report (no extension)

# Substitution: ${VAR/pattern/replacement}
echo "${FILE/alice/bob}"    # /home/bob/documents/report.txt (first only)
echo "${FILE//o/o}"         # /home/alice/documents/report.txt (all)

# Case conversion (bash 4.0+)
echo "${NAME^}"             # Alice (capitalize first)
echo "${NAME^^}"            # ALICE (all uppercase)
lower="HELLO"
echo "${lower,,}"           # hello (all lowercase)
echo "${lower,}"            # hELLO (lowercase first char)
```

Special Variables

```
#!/usr/bin/env bash
# Run this as: ./script.sh arg1 arg2 arg3

echo "=== Positional Parameters ==="
echo "Script name:      $0"      # ./script.sh
echo "First arg:       $1"       # arg1
echo "Second arg:      $2"       # arg2
echo "All args (\$@):   \$@"       # arg1 arg2 arg3
echo "All args (\$*):   \$*"       # arg1 arg2 arg3 (subtle difference!)
echo "Arg count:       $#"       # 3

echo ""
echo "=== Process Information ==="
echo "Script PID:      $$"       # Current process ID
echo "Last BG PID:     $!"       # PID of last background command
echo "Shell options:    $-"       # Active shell options

echo ""
echo "=== Exit Status ==="
ls /tmp >/dev/null 2>&1
echo "ls exit code:    $?"       # 0 (success)
ls /nonexistent 2>/dev/null
echo "ls fail code:    $?"       # 2 (failure – non-zero)

# $@ vs $* – the critical difference:
show_args() {
    echo "With \$@:"
    for arg in "$@"; do echo "  → '$arg'"; done # Preserves spaces!
```

```
    echo "With \${*}:"  
    for arg in "$*"; do echo "  → '$arg'"; done # Treats all as ONE string!  
}  
show_args "hello world" "foo bar" # Note: two args with spaces
```

Common Data Types in Bash

Strings

```
#!/usr/bin/env bash

# — String Declaration —————
SINGLE='No $expansion here, literal \n' # Single quotes: NO expansion
DOUBLE="Expansion works: $USER"       # Double quotes: expansion happens
NO_QUOTES=NoSpacesNeeded              # OK without quotes if no spaces

echo "$SINGLE"
echo "$DOUBLE"

# — Multi-line strings —————
MULTI_LINE="Line 1
Line 2
Line 3"
echo "$MULTI_LINE"

# Using heredoc for cleaner multi-line:
read -r -d '' HEREDOC_STRING << 'MARKER'
This is line 1
This is line 2
Variables like $HOME are NOT expanded (notice the quotes around MARKER)
MARKER
echo "$HEREDOC_STRING"

# WITH expansion (no quotes around marker):
read -r -d '' HEREDOC_EXPANDED << MARKER
```

```
User home is: $HOME
Today is: $(date +%Y-%m-%d)
MARKER
echo "$HEREDOC_EXPANDED"

# — String comparison —————
A="hello"
B="world"
C="hello"

[[ "$A" == "$C" ]] && echo "A equals C"          # True
[[ "$A" != "$B" ]] && echo "A differs from B" # True
[[ "$A" < "$B" ]] && echo "A is before B alphabetically" # True
[[ -z "" ]] && echo "Empty string"              # True
[[ -n "text" ]] && echo "Non-empty string"       # True

# Pattern matching in [[ ]]
[[ "$A" == h* ]] && echo "Starts with h"         # True (glob pattern)
[[ "$A" =~ ^hel ]] && echo "Matches regex"      # True (regex!)
```

Integers and Arithmetic

```
#!/usr/bin/env bash

# — Integer Declaration —————
declare -i NUM=42      # Explicitly integer – arithmetic on assignment
COUNT=0              # Still a string, but can do arithmetic

# — Arithmetic: $(( )) —————
echo $((2 + 3))        # 5
echo $((10 - 4))       # 6
echo $((3 * 7))        # 21
echo $((15 / 4))       # 3 (integer division – no decimals!)
echo $((15 % 4))       # 3 (modulo/remainder)
echo $((2 ** 8))       # 256 (exponentiation)

# Assignment in arithmetic:
((COUNT++))          # Increment
((COUNT += 5))       # Add 5
((COUNT *= 2))       # Multiply by 2
echo "Count: $COUNT" # 12

# Variables in arithmetic:
A=10
B=3
echo "Sum: $((A + B))" # 13
echo "Product: $((A * B))" # 30

# Arithmetic return value (0=true, 1=false – OPPOSITE of normal!)
if (( A > B )); then
    echo "$A is greater than $B"
```

fi

Floating point? Bash can't do it – use bc or awk:

echo "scale=2; 10 / 3" | bc # 3.33

awk 'BEGIN { printf "%.4f\n", 10/3 }' # 3.3333

Arrays (Indexed)

```
#!/usr/bin/env bash
```

```
# — Array Declaration —————
```

```
# Method 1: Direct declaration
```

```
FRUITS=("apple" "banana" "cherry" "date")
```

```
# Method 2: declare -a
```

```
declare -a COLORS
```

```
COLORS[0]="red"
```

```
COLORS[1]="green"
```

```
COLORS[2]="blue"
```

```
# Method 3: From command output
```

```
FILES=($(ls /tmp/*.log 2>/dev/null)) # Array of .log files
```

```
# — Array Access —————
```

```
echo "${FRUITS[0]}"           # apple (first element)
```

```
echo "${FRUITS[1]}"           # banana
```

```
echo "${FRUITS[-1]}"          # date (LAST element – bash 4.3+)
```

```
echo "${FRUITS[@]}"           # All elements: apple banana cherry date
```

```
echo "${FRUITS[*]}"           # All elements (subtle difference with quotes)
```

```
echo "${#FRUITS[@]}"          # 4 (array length)
```

```
echo "${!FRUITS[@]}"          # 0 1 2 3 (array indices)
```

```
# — Array Modification —————
```

```
FRUITS+=("elderberry")        # Append one element
```

```
FRUITS+=("fig" "grape")        # Append multiple
```

```
# Remove an element
```

```
unset FRUITS[2]          # Remove cherry (index 2 – creates a gap!)
echo "${FRUITS[@]}"      # apple banana date elderberry fig grape

# — Array Slicing —————
echo "${FRUITS[@]:1:3}"  # banana date elderberry (start:count)

# — Iterating Arrays —————
echo "=== Iterating with indices ==="
for i in "${!FRUITS[@]}"; do
    echo "  Index $i: ${FRUITS[$i]}"
done

echo "=== Iterating elements ==="
for fruit in "${FRUITS[@]}"; do
    echo "  Fruit: $fruit"
done

# — Practical: Check if value is in array —————
contains() {
    local needle="$1"
    shift
    local element
    for element in "$@"; do
        [[ "$element" == "$needle" ]] && return 0
    done
    return 1
}

if contains "banana" "${FRUITS[@]}"; then
    echo "Found banana!"
fi
```

Associative Arrays (Dictionaries)

```
#!/usr/bin/env bash
# Requires bash 4.0+

# — Associative Array Declaration (MUST use declare -A) —————
declare -A PERSON
PERSON["name"]="Alice Smith"
PERSON["age"]=30
PERSON["role"]="Engineer"
PERSON["city"]="San Francisco"

# Or initialize all at once:
declare -A CONFIG=(
    ["host"]="localhost"
    ["port"]="5432"
    ["database"]="myapp"
    ["user"]="dbuser"
)

# — Access —————
echo "${PERSON["name"]}"      # Alice Smith
echo "${CONFIG["port"]}"     # 5432

# — All keys and values —————
echo "Keys:  ${!PERSON[@]}"   # name age role city
echo "Values: ${PERSON[@]}"   # Alice Smith 30 Engineer San Francisco

# — Iterate —————
echo "=== Person details ==="
for key in "${!PERSON[@]}; do
```

```
    printf "  %-10s: %s\n" "$key" "${PERSON[$key]}"
done

# — Check if key exists —————
if [[ -v PERSON["name"] ]]; then
    echo "Key 'name' exists"
fi

# — Practical: Word frequency counter —————
declare -A WORD_COUNT
while read -r word; do
    ((WORD_COUNT[$word]++)) # Increment count for each word
done < <(tr ' ' '\n' <<< "the cat sat on the mat the cat")

echo "=== Word Frequencies ==="
for word in "${!WORD_COUNT[@]}"; do
    printf "  '%s': %d\n" "$word" "${WORD_COUNT[$word]}"
done | sort -t: -k2 -rn
```

Environment Variables

```
#!/usr/bin/env bash

# — View environment variables —————
env                # All environment variables
printenv HOME      # Specific variable
echo "$PATH"       # PATH variable

# — Export: make variable available to child processes —————
MY_VAR="local only"    # Only in this script

export MY_VAR_GLOBAL="visible to children"
export -n MY_VAR        # Remove from export (unexport)

# Test:
MY_SECRET="secret"
bash -c 'echo "Secret: $MY_SECRET"'    # Empty! Not exported.
export MY_SECRET
bash -c 'echo "Secret: $MY_SECRET"'    # secret! Now exported.

# — Common important environment variables —————
echo "Home:    $HOME"        # /home/alice
echo "User:    $USER"        # alice
echo "Shell:   $SHELL"       # /bin/bash
echo "Path:    $PATH"        # /usr/bin:/bin:...
echo "Editor:  ${EDITOR:-vi}" # Default editor
echo "Term:    $TERM"        # Terminal type (xterm-256color)
echo "Lang:    $LANG"        # en_US.UTF-8
echo "PWD:     $PWD"         # Current directory
echo "OLDPWD:  $OLDPWD"      # Previous directory
```

```
# — Modify PATH —————  
# Add directory to PATH (prepend for priority)  
export PATH="$HOME/bin:$PATH"  
  
# Add at end (lower priority)  
export PATH="$PATH:/opt/myapp/bin"  
  
# Make it permanent (add to ~/.bashrc or ~/.bash_profile)  
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc
```

Practical Examples for Variables

Example 1: Comprehensive Variable Demo Script

```
#!/usr/bin/env bash
# variables_demo.sh - Complete variable showcase
set -euo pipefail

# — Configuration with defaults —————
LOG_LEVEL="${LOG_LEVEL:-INFO}"      # Override with env var
MAX_RETRY="${MAX_RETRY:-3}"
OUTPUT_DIR="${OUTPUT_DIR:-/tmp/output}"

# — Validate numeric variables —————
is_integer() {
    [[ "$1" =~ ^[0-9]+$ ]]
}

if ! is_integer "$MAX_RETRY"; then
    echo "ERROR: MAX_RETRY must be a number, got: $MAX_RETRY" >&2
    exit 1
fi

# — String manipulation showcase —————
URL="https://api.example.com/v1/users/alice?format=json"

echo "=== URL Parsing Without External Tools ==="
# Extract protocol
PROTOCOL="${URL%%:.*}"                # https
# Remove protocol
```

```
NO_PROTO="${URL#*://}"          # api.example.com/v1/users/alice?format=json
# Extract hostname
HOSTNAME="${NO_PROTO%/*}"        # api.example.com
# Extract path
PATH_ONLY="${NO_PROTO#*/}"       # v1/users/alice?format=json
# Extract query string
QUERY="${URL##*\?}"             # format=json
# Filename from path
ENDPOINT="${URL##*/}"           # alice?format=json
ENDPOINT="${ENDPOINT%%\?*}"      # alice

printf "Protocol:  %s\n" "$PROTOCOL"
printf "Hostname:  %s\n" "$HOSTNAME"
printf "Path:      %s\n" "$PATH_ONLY"
printf "Query:     %s\n" "$QUERY"
printf "Endpoint:  %s\n" "$ENDPOINT"
```

Example 2: Configuration File Parser

```
#!/usr/bin/env bash
# parse_config.sh – Read key=value config files into variables

parse_config() {
    local config_file="$1"
    local -n config_array="$2"    # nameref – pass array by reference (bash 4.3+)

    if [[ ! -f "$config_file" ]]; then
        echo "Config file not found: $config_file" >&2
        return 1
    fi

    while IFS='=' read -r key value; do
        # Skip comments and blank lines
        [[ "$key" =~ ^[:space:]*# ]] && continue
        [[ -z "$key" ]] && continue

        # Trim whitespace
        key="${key// /}"
        value="${value# }"    # Remove leading space
        value="${value%% *}"  # Remove trailing comment

        config_array["$key"]="$value"
    done < "$config_file"
}

# — Create a sample config file —————
cat > /tmp/app.conf << 'EOF'
# Application Configuration
```

```
host = localhost
port = 8080
database = myapp
# This is a comment
debug = false
max_connections = 100
EOF

# — Parse and use it —————
declare -A APP_CONFIG
parse_config "/tmp/app.conf" APP_CONFIG

echo "=== Parsed Configuration ==="
for key in "${!APP_CONFIG[@]}"; do
    printf " %-20s = %s\n" "$key" "${APP_CONFIG[$key]}"
done

# Access specific values
echo ""
echo "Server: ${APP_CONFIG[host]}:${APP_CONFIG[port]}"
echo "DB:      ${APP_CONFIG[database]}"
```



Hands-on Exercise 2.2

```
# EXERCISE: Variable Mastery

# Task 1: String operations without external tools
FULL_PATH="/var/log/nginx/access.log"
# a) Extract just the filename (access.log)
FILENAME="${FULL_PATH##*/}"
echo "Filename: $FILENAME"

# b) Extract just the directory (/var/log/nginx)
DIRNAME="${FULL_PATH%/*}"
echo "Directory: $DIRNAME"

# c) Extract the extension (log)
EXTENSION="${FILENAME##*.}"
echo "Extension: $EXTENSION"

# d) Filename without extension (access)
BASENAME="${FILENAME%.*}"
echo "Base name: $BASENAME"

# Task 2: Arithmetic calculator
read -p "Enter first number: " NUM1
read -p "Enter second number: " NUM2

echo "Sum:      $((NUM1 + NUM2))"
echo "Diff:     $((NUM1 - NUM2))"
echo "Product:   $((NUM1 * NUM2))"
echo "Division: $(echo "scale=2; $NUM1/$NUM2" | bc)"
```

```
# Task 3: Array operations
# Create an array of the first 10 prime numbers
PRIMES=(2 3 5 7 11 13 17 19 23 29)
echo "First prime: ${PRIMES[0]}"
echo "Last prime:  ${PRIMES[-1]}"
echo "Count:       ${#PRIMES[@]}"
echo "Sum: $((PRIMES[0]+PRIMES[1]+PRIMES[2]+PRIMES[3]+PRIMES[4]))"

# Task 4: Special variables
echo "This script is: $0"
echo "Your shell is:  $SHELL"
echo "Your home is:   $HOME"
echo "Script PID:     $$"
```

Section 2.3: Control Structures

Learning Objectives

- Write correct `if/elif/else` conditional statements
- Use `case` statements for multi-branch logic
- Implement `for`, `while`, and `until` loops
- Use `break` and `continue` for loop control
- Understand test expressions (`[[ ]]`, `(( ))`, file tests)

Introduction

Control structures are the decision-making brain of your script. Without them, a script is just a list of commands run top-to-bottom. With conditionals and loops, you can make scripts that adapt to different situations, process collections of items, retry failed operations, and respond to user input.

Conditional Statements

The `if` Statement

```
#!/usr/bin/env bash
```

```
# — Syntax
```

```
if CONDITION; then
```

```
    # commands when condition is TRUE (exit code 0)
```

```
elif ANOTHER_CONDITION; then
```

```
    # commands when second condition is true
```

```
else
```

```
    # commands when ALL conditions are false
```

```
fi
```

```
# — IMPORTANT: In bash, 0 = success = TRUE, non-zero = failure = FALSE
```

```
# This is OPPOSITE to most programming languages!
```

```
# — Simple if
```

```
AGE=20
```

```
if [[ $AGE -ge 18 ]]; then
```

```
    echo "Adult"
```

```
fi
```

```
# — if-else
```

```
hour=$(date +%H) # Current hour (00-23)
```

```
if [[ $hour -lt 12 ]]; then
```

```
    echo "Good morning!"
```

```
elif [[ $HOUR -lt 17 ]]; then
    echo "Good afternoon!"
elif [[ $HOUR -lt 20 ]]; then
    echo "Good evening!"
else
    echo "Good night!"
fi
```

```
# — One-line if (useful for simple checks) —————
[[ -f "/etc/passwd" ]] && echo "passwd file exists"
[[ -d "/nonexistent" ]] || echo "Directory not found"
```

Test Expressions — The Complete Reference

— FILE TESTS

```
[[ -f "$path" ]] # Is a regular FILE?
[[ -d "$path" ]] # Is a DIRECTORY?
[[ -e "$path" ]] # EXISTS (file or dir)?
[[ -L "$path" ]] # Is a symbolic LINK?
[[ -r "$path" ]] # Is READABLE?
[[ -w "$path" ]] # Is WRITABLE?
[[ -x "$path" ]] # Is EXECUTABLE?
[[ -s "$path" ]] # Has SIZE > 0 (non-empty)?
[[ -z "$path" ]] # Is ZERO length (empty string)?
[[ -n "$path" ]] # Is NON-zero length (non-empty string)?
```

File comparison:

```
[[ file1 -nt file2 ]] # file1 is NEWER than file2
[[ file1 -ot file2 ]] # file1 is OLDER than file2
[[ file1 -ef file2 ]] # Same inode (hard links to same file)
```

— STRING TESTS

```
[[ "$A" == "$B" ]] # Equal strings
[[ "$A" != "$B" ]] # Not equal
[[ "$A" < "$B" ]] # Alphabetically before (in [[ ]])
[[ "$A" > "$B" ]] # Alphabetically after
[[ "$A" =~ regex ]] # Matches extended regular expression
[[ "$A" == patt* ]] # Glob pattern match
```

— NUMERIC TESTS

```
[[ $A -eq $B ]] # Equal (numbers) – or use (( A == B ))
[[ $A -ne $B ]] # Not equal
[[ $A -lt $B ]] # Less than
```

```
[[ $A -le $B ]]      # Less than or equal
[[ $A -gt $B ]]      # Greater than
[[ $A -ge $B ]]      # Greater than or equal

# — LOGICAL OPERATORS —————
[[ COND1 && COND2 ]] # Both true (AND)
[[ COND1 || COND2 ]] # Either true (OR)
[[ ! COND ]]        # NOT

# — ARITHMETIC in if —————
if (( 5 > 3 )); then echo "math works"; fi
if (( AGE >= 18 && AGE < 65 )); then echo "working age"; fi
```

Practical if Examples

```
#!/usr/bin/env bash
# smart_check.sh - Practical conditionals

# — Check if running as root —————
check_root() {
    if [[ $EUID -ne 0 ]]; then
        echo "❌ This script requires root privileges." >&2
        echo "    Run: sudo $0" >&2
        exit 1
    fi
    echo "✅ Running as root"
}

# — Check if command exists —————
require_command() {
    local cmd="$1"
    if ! command -v "$cmd" &>/dev/null; then
        echo "❌ Required command '$cmd' not found." >&2
        echo "    Install it with: sudo apt install $cmd" >&2
        exit 1
    fi
    echo "✅ Found: $cmd ($(command -v "$cmd"))"
}

# — Check disk space —————
check_disk_space() {
    local mount_point="${1:-/}"
    local threshold="${2:-90}"      # Alert if above 90%
}
```

```

local usage
usage=$(df "$mount_point" | awk 'NR==2 {print $5}' | tr -d '%')

if [[ $usage -ge $threshold ]]; then
    echo "⚠️ WARNING: $mount_point is ${usage}% full!"
    return 1
elif [[ $usage -ge $((threshold - 10)) ]]; then
    echo "📊 NOTICE: $mount_point is ${usage}% full (approaching limit)"
    return 0
else
    echo "✅ $mount_point usage: ${usage}% (healthy)"
    return 0
fi
}

```

```

# — File type checker —————
describe_file() {
    local path="$1"

    if [[ ! -e "$path" ]]; then
        echo "'$path' does not exist"
    elif [[ -d "$path" ]]; then
        local count
        count=$(ls "$path" | wc -l)
        echo "'$path' is a directory containing $count items"
    elif [[ -L "$path" ]]; then
        echo "'$path' is a symlink pointing to: $(readlink "$path")"
    elif [[ -f "$path" ]]; then
        local size
        size=$(du -sh "$path" | cut -f1)
        echo "'$path' is a file ($size)"
    else

```

```
    echo "'$path' is a special file"
fi
}
```

Test them:

```
require_command "bash"
require_command "grep"
check_disk_space "/" 95
describe_file "/etc/passwd"
describe_file "/tmp"
describe_file "/bin/ls"
```

The `case` Statement

```
#!/usr/bin/env bash

# case is bash's equivalent of switch – perfect for multi-choice logic

# — Basic syntax —————

FRUIT="apple"

case "$FRUIT" in
    apple)
        echo "It's an apple 🍏"
        ;;
    banana|plantain)    # Match EITHER banana OR plantain
        echo "It's a banana/plantain 🍌"
        ;;
    cherry|grape|berry*) # Glob patterns work!
        echo "It's a small fruit 🍒"
        ;;
    *)                  # Default (catch-all)
        echo "Unknown fruit: $FRUIT"
        ;;
esac

# — Practical: Command dispatcher —————

ACTION="${1:-help}"

case "$ACTION" in
    start|begin)
        echo "Starting the service..."
        # systemctl start myapp
        ;;
```

```
stop|halt|kill)
    echo "Stopping the service..."
    # systemctl stop myapp
    ;;
restart|reload)
    echo "Restarting the service..."
    # systemctl restart myapp
    ;;
status|check)
    echo "Checking status..."
    # systemctl status myapp
    ;;
help|--help|-h|"")
    echo "Usage: $0 {start|stop|restart|status}"
    ;;
*)
    echo "Unknown action: $ACTION" >&2
    echo "Usage: $0 {start|stop|restart|status}" >&2
    exit 1
    ;;
esac

# — Case with file extensions —————
process_file() {
    local file="$1"
    local ext="${file##*}."

    case "${ext,,}" in
        jpg|jpeg|png|gif|webp)
            echo "Image file: $file"
            # convert "$file" compressed/"$file"
            ;;
    esac
}
```

```
mp4|avi|mkv|mov)
    echo "Video file: $file"
    ;;
txt|md|rst)
    echo "Text file: $file ($(wc -l < "$file") lines)"
    ;;
sh|bash)
    echo "Shell script: $file"
    bash -n "$file" && echo " Syntax OK" || echo " Syntax ERROR!"
    ;;
*)
    echo "Unknown type: $file"
    ;;
esac
}

process_file "/etc/passwd"
process_file "/usr/bin/bash"
```

Looping Constructs

The `for` Loop

```
#!/usr/bin/env bash
```

```
# — FORM 1: List iteration (most common) —————
```

```
for item in apple banana cherry; do
    echo "Fruit: $item"
done
```

```
# — FORM 2: Range iteration —————
```

```
for i in {1..5}; do
    echo "Number: $i"
done
```

```
# With step:
```

```
for i in {0..20..5}; do    # 0, 5, 10, 15, 20
    echo "Step: $i"
done
```

```
# — FORM 3: C-style for loop —————
```

```
for ((i = 0; i < 5; i++)); do
    echo "C-style: $i"
done
```

```
# — FORM 4: Iterate over array —————
```

```
SERVERS=("web01" "web02" "db01" "cache01")
for server in "${SERVERS[@]}"; do
    echo "Checking server: $server"
```

```
# ssh "$server" uptime
done

# — FORM 5: Iterate over files —————
for file in /etc/*.conf; do
    [[ -f "$file" ]] || continue    # Skip if not a file
    echo "Config: $(basename "$file") ($(wc -l < "$file") lines)"
done

# — FORM 6: Iterate over command output —————
for user in $(awk -F: '$3 >= 1000 && $3 < 65534 {print $1}' /etc/passwd); do
    echo "Regular user: $user (home: $(getent passwd "$user" | cut -d: -f6))"
done

# — Nested loops —————
echo "=== Multiplication Table ==="
for ((row=1; row<=5; row++)); do
    for ((col=1; col<=5; col++)); do
        printf "%4d" $((row * col))
    done
done
echo ""
done
```

The while Loop

```
#!/usr/bin/env bash
```

```
# — Basic while —————
```

```
COUNT=1
while [[ $COUNT -le 5 ]]; do
    echo "Count: $COUNT"
    ((COUNT++))
done
```

```
# — Reading file line by line (the RIGHT way) —————
```

```
while IFS= read -r line; do
    echo "Line: $line"
done < /etc/hostname
```

```
# — Reading multiple fields per line —————
```

```
while IFS=: read -r username _ uid gid gecos home shell; do
    # Only show regular users (UID >= 1000)
    if [[ $uid -ge 1000 && $uid -lt 65534 ]]; then
        printf "User: %-15s Home: %s\n" "$username" "$home"
    fi
done < /etc/passwd
```

```
# — Infinite loop with break —————
```

```
ATTEMPTS=0
MAX_ATTEMPTS=3

while true; do
    ((ATTEMPTS++))
    echo "Attempt $ATTEMPTS of $MAX_ATTEMPTS..."

```

```
if ping -c1 -W1 google.com &>/dev/null; then
    echo "✅ Network is up!"
    break
fi

if [[ $ATTEMPTS -ge $MAX_ATTEMPTS ]]; then
    echo "❌ Network check failed after $MAX_ATTEMPTS attempts"
    exit 1
fi

echo "Waiting 5 seconds before retry..."
sleep 5
done

# — Read from stdin (pipe-friendly) —————
# This reads lines piped into the script
# echo -e "line1\nline2\nline3" | while IFS= read -r line; do
#     echo "Processing: $line"
# done

# — Countdown timer —————
countdown() {
    local seconds="$1"
    while [[ $seconds -gt 0 ]]; do
        printf "\rCountdown: %3d seconds remaining..." "$seconds"
        sleep 1
        ((seconds--))
    done
    printf "\rCountdown: Done!          \n"
```

```
}
```

```
countdown 5
```

The `until` Loop

```
#!/usr/bin/env bash

# until: opposite of while – loops UNTIL condition becomes TRUE

# — Basic until —————
COUNT=1
until [[ $COUNT -gt 5 ]]; do    # Keep going UNTIL count exceeds 5
    echo "Count: $COUNT"
    ((COUNT++))
done

# — Practical: Wait for service to start —————
wait_for_port() {
    local host="$1"
    local port="$2"
    local timeout="${3:-30}"
    local elapsed=0

    echo "Waiting for $host:$port to be available..."

    until nc -z "$host" "$port" 2>/dev/null; do
        if [[ $elapsed -ge $timeout ]]; then
            echo "❌ Timeout after ${timeout}s waiting for $host:$port" >&2
            return 1
        fi
        sleep 1
        ((elapsed++))
        printf "  Waiting... %ds\r" "$elapsed"
    done
```

```
echo "✅ $host:$port is available! (after ${elapsed}s)"  
}
```

```
# Usage: wait_for_port localhost 5432 60
```

break and continue

```
#!/usr/bin/env bash
```

```
# — break: exit the loop entirely —————
```

```
echo "=== break example ==="
```

```
for i in {1..10}; do
    if [[ $i -eq 6 ]]; then
        echo "Breaking at $i"
        break
    fi
    echo "i = $i"
done
```

```
# — continue: skip to next iteration —————
```

```
echo "=== continue example (odd numbers only) ==="
```

```
for i in {1..10}; do
    if (( i % 2 == 0 )); then
        continue # Skip even numbers
    fi
    echo "Odd: $i"
done
```

```
# — break/continue in nested loops —————
```

```
echo "=== Nested loop with break 2 ==="
```

```
for i in 1 2 3; do
    for j in A B C; do
        if [[ $i -eq 2 && $j == "B" ]]; then
            echo "Breaking outer loop at i=$i, j=$j"
            break 2 # break 2 = exit 2 levels of loops!
        fi
    done
done
```

```
        echo " i=$i, j=$j"
    done
done
echo "After nested loop"

# — Practical: Find first available port —————
find_free_port() {
    local start_port="${1:-8000}"
    local end_port="${2:-9000}"

    for port in $(seq "$start_port" "$end_port"); do
        if ! ss -tlnp 2>/dev/null | grep -q ":$port "; then
            echo "$port"
            return 0
        fi
    done
    echo "No free port found in range $start_port-$end_port" >&2
    return 1
}

FREE_PORT=$(find_free_port 8080 8090)
echo "First free port: $FREE_PORT"
```

Practical Control Structure Examples

Example 1: Retry Mechanism

```
#!/usr/bin/env bash
# retry.sh – Retry a command up to N times with exponential backoff

retry() {
    local max_attempts="$1"
    local delay="${2:-1}"
    shift 2
    local cmd=("$@")

    local attempt=1

    while [[ $attempt -le $max_attempts ]]; do
        echo "Attempt $attempt/$max_attempts: ${cmd[*]}"

        if "${cmd[@]}"; then
            echo "✅ Success on attempt $attempt"
            return 0
        fi

        if [[ $attempt -lt $max_attempts ]]; then
            echo "⚠️ Failed. Retrying in ${delay}s..."
            sleep "$delay"
            delay=$((delay * 2)) # Exponential backoff
        fi

        ((attempt++))
    done
}
```

```
done

echo "❌ All $max_attempts attempts failed" >&2
return 1
}

# Usage:
retry 3 2 curl -sf https://example.com -o /dev/null
```

Example 2: Menu System

```
#!/usr/bin/env bash
# menu.sh – Interactive menu system

show_menu() {
    clear
    echo "┌───────────────────────────────────┐"
    echo "│          SYSTEM ADMIN MENU          │"
    echo "└───────────────────────────────────┘"
    echo "  1) System Information                │"
    echo "  2) Disk Usage                       │"
    echo "  3) Running Processes (top 10)       │"
    echo "  4) Network Connections              │"
    echo "  5) Check Disk Health                │"
    echo "  q) Quit                             │"
    echo "└───────────────────────────────────┘"
    echo ""
    printf "Enter choice: "
}

run_option() {
    local choice="$1"
    case "$choice" in
        1)
            echo ""
            echo "=== System Information ==="
            uname -a
            uptime
            ;;
        2)
```

```
    echo ""
    echo "=== Disk Usage ==="
    df -h
    ;;
3)
    echo ""
    echo "=== Top 10 Processes by CPU ==="
    ps aux --sort=-%cpu | head -11
    ;;
4)
    echo ""
    echo "=== Active Network Connections ==="
    ss -tuln
    ;;
5)
    echo ""
    echo "=== Available Disk Devices ==="
    lsblk
    ;;
q|Q|quit|exit)
    echo "Goodbye!"
    exit 0
    ;;
*)
    echo "❌ Invalid option: '$choice'"
    ;;
esac
}

# — Main loop —————
while true; do
    show_menu
```

```
read -r user_choice
run_option "$user_choice"
echo ""
read -rp "Press Enter to continue..."
done
```

Example 3: Bulk File Processor

```
#!/usr/bin/env bash
# bulk_process.sh – Process multiple files with progress tracking

process_files() {
    local source_dir="${1:-.}"
    local pattern="${2:-*.txt}"
    local processed=0
    local failed=0
    local skipped=0

    # Get total count
    local total
    total=$(find "$source_dir" -maxdepth 1 -name "$pattern" -type f | wc -l)

    if [[ $total -eq 0 ]]; then
        echo "No files found matching: $source_dir/$pattern"
        return
    fi

    echo "Found $total files to process..."
    echo ""

    # Process each file
    for file in "$source_dir"/$pattern; do
        [[ -f "$file" ]] || continue

        local filename
        filename=$(basename "$file")
        local current=$((processed + failed + skipped + 1))
```

```
# Progress bar
printf "[%3d/%3d] Processing: %-30s" "$current" "$total" "$filename"

# Check if file is empty
if [[ ! -s "$file" ]]; then
    echo "SKIPPED (empty)"
    ((skipped++))
    continue
fi

# Do the actual processing (example: count words)
local word_count
if word_count=$(wc -w < "$file" 2>/dev/null); then
    echo "OK ($word_count words)"
    ((processed++))
else
    echo "FAILED"
    ((failed++))
fi
done

echo ""
echo "=== Summary ==="
echo "  Processed: $processed"
echo "  Failed:    $failed"
echo "  Skipped:   $skipped"
echo "  Total:     $total"
}

# Create test files
mkdir -p /tmp/test_files
```

```
for i in {1..5}; do
    echo "File $i content with $((RANDOM % 100)) words" > "/tmp/test_files/file${i}.txt"
done

touch /tmp/test_files/empty.txt    # Empty file

process_files /tmp/test_files "*.txt"
```



Hands-on Exercise 2.3

```
# EXERCISE: Control Structures

# Task 1: Grade Calculator
# Write a case statement that converts a numeric score to a letter grade
grade_calculator() {
    local score="$1"
    local grade

    if (( score >= 90 )); then
        grade="A"
    elif (( score >= 80 )); then
        grade="B"
    elif (( score >= 70 )); then
        grade="C"
    elif (( score >= 60 )); then
        grade="D"
    else
        grade="F"
    fi

    echo "Score $score = Grade $grade"
}

for score in 95 82 73 65 45; do
    grade_calculator "$score"
done

# Task 2: FizzBuzz with for loop
```

```
echo "=== FizzBuzz 1-30 ==="
for i in {1..30}; do
    if (( i % 15 == 0 )); then
        echo "FizzBuzz"
    elif (( i % 3 == 0 )); then
        echo "Fizz"
    elif (( i % 5 == 0 )); then
        echo "Buzz"
    else
        echo "$i"
    fi
done

# Task 3: Read /etc/passwd and list home directories that exist
echo "=== Users with existing home directories ==="
while IFS=: read -r username _ uid _ _ home _; do
    if (( uid >= 1000 && uid < 65534 )) && [[ -d "$home" ]]; then
        echo "  $username → $home ($(du -sh "$home" 2>/dev/null | cut -f1))"
    fi
done < /etc/passwd

# Task 4: Pattern matching with case
for file in /etc/hosts /usr/bin/bash /tmp /dev/null; do
    case "$file" in
        /etc/*)    echo "$file is a configuration file" ;;
        /usr/bin/*) echo "$file is a binary" ;;
        /tmp/*)    echo "$file is temporary" ;;
        /dev/*)    echo "$file is a device" ;;
        *)         echo "$file is somewhere else" ;;
    esac
done
```

[End of Part 1 — Sections 2.1 through 2.3]